Module 14 - Python Typing Type Hints & Annotations

Table of Contents

- [ ] 1. Python is Dynamically Typed
- [ ] 2. Variable Annotations
- [ ] 3. Function Annotations
- [ ] 4. Complex Types: List, Dict, Tuple, Set
- [ ] 5. Optional - Values That Can Be None
- [ ] 6. Union - Values That Can Be Multiple Types
- [ ] 7. Type Aliases
- [ ] 8. Why Type Hints Matter in Cybersecurity Code
- [ ] 9. Summary

---

1. Python is Dynamically Typed

Analogy: Imagine a locker at a security operations centre. In Python's world, that locker can hold anything - a laptop, a USB drive, a stack of incident reports - and you can swap the contents any time. In stricter languages (like Java or C#), each locker is labelled and can only ever hold one specific type of item.

Python checks types **at runtime** (when the code actually runs), not before. This makes Python flexible, but it also means bugs caused by wrong types only show up when the line is executed - which could be deep inside a scan that runs at 2 AM.

Dynamic typing in action

Python lets you change the type of a variable freely:

In [ ]:
```
# Dynamic typing: Python does NOT lock the type of a variable
cvss_score = 9.8           # starts as float
print(type(cvss_score))    # <class 'float'>

cvss_score = "CRITICAL"    # now it's a string - Python allows this!
print(type(cvss_score))    # <class 'str'>
```

The same thing in Java (a statically typed language)

```
double cvssScore = 9.8;
cvssScore = "CRITICAL";  // ERROR: incompatible types
```

Java refuses to compile - `cvssScore` was declared as `double`, it cannot become a `String`.

Duck typing

Python also uses **duck typing**: "*If it walks like a duck and quacks like a duck, it IS a duck.*"

In practice: Python doesn't care what **type** an object is, it only cares whether the object **has the method or attribute you're trying to use**.

In [ ]:
```
# Duck typing example: any object with a __len__ method works with len()

class FirewallRuleset:
    """Represents a set of firewall rules for a network segment."""
    def __init__(self, rules: list):
        self.rules = rules

    def __len__(self):
        return len(self.rules)  # how many rules are in this ruleset


perimeter_rules = FirewallRuleset(["ALLOW 443", "ALLOW 80", "DENY ALL"])

# len() works on strings, lists, dicts... and our custom class
print(len("192.168.1.1"))     # 11  - works on str
print(len([22, 80, 443]))      # 3   - works on list
print(len(perimeter_rules))    # 3   - works on FirewallRuleset too!
```

Practice - Section 1

Predict: Before running the cell below, predict what each `print` outputs.

```
port = 443
print(type(port))
port = "HTTPS"
print(type(port))
port = [443, 8443]
print(type(port))
```

Write your predictions in a comment, then run the cell to verify.

In [ ]:
```
# Your predictions:
# print 1: ?
# print 2: ?
# print 3: ?

# Your code here - run the block and check
port = 443
print(type(port))
```

```
port = "HTTPS"
print(type(port))
port = [443, 8443]
print(type(port))
```

▶ 💡 **SOLUTION**

---

2. Variable Annotations

Analogy: Imagine labelling every drawer in your SOC analyst's desk: "*This drawer holds IP addresses (strings)*", "*This one holds CVSS scores (floats)*". The labels don't lock the drawer - you could still put anything in - but they communicate intent to everyone on the team.

Type annotations work exactly like that. They are **hints**, not enforcement. Python still runs the code even if the wrong type is stored.

Syntax

`variable_name: type = value`

Without annotation	With annotation
`port = 443`	`port: int = 443`
`target_ip = "10.0.0.1"`	`target_ip: str = "10.0.0.1"`
`firewall_enabled = True`	`firewall_enabled: bool = True`
`cvss_score = 9.8`	`cvss_score: float = 9.8`

In [ ]:
```
# Variable annotations: documenting expected types

target_ip: str = "192.168.1.100"    # IP to scan
ssh_port: int = 22                   # SSH default port
cvss_score: float = 9.8              # CRITICAL severity threshold
firewall_enabled: bool = True        # is perimeter firewall active?
failed_attempts: int = 0             # brute force counter

print(target_ip)
print(ssh_port)
print(cvss_score)
print(firewall_enabled)
```

Important: annotations are NOT enforced

Python will NOT raise an error if you store the wrong type. It's a documentation tool. To actually catch type errors, you need a tool called **mypy** (covered in Section 8).

In [ ]:
```
# Annotations do NOT prevent wrong types at runtime
cvss_score: float = 9.8
print(f"Score: {cvss_score}, type: {type(cvss_score)}")

cvss_score = "CRITICAL"   # Python allows this despite the : float annotation
print(f"Score: {cvss_score}, type: {type(cvss_score)}")
# No error! The annotation is just a hint for developers and tools.
```

Practice - Section 2

Write: Annotate the 5 variables below with the correct types. Each variable represents a field in a security incident report.

- `incident_id` - a text identifier like `"INC-2024-0042"`
- `severity_level` - a whole number from 1 to 5
- `affected_hosts` - a count of machines impacted
- `is_contained` - whether the threat is under control
- `risk_score` - a decimal score like 7.3

In [ ]:
```
# Your code here - add type annotations to each variable
incident_id = "INC-2024-0042"
severity_level = 4
affected_hosts = 13
is_contained = False
risk_score = 7.3

print(incident_id, severity_level, affected_hosts, is_contained, risk_score)
```

▶ 💡 **SOLUTION**

---

3. Function Annotations

Analogy: A function's signature is like a job description. Without annotations, the job posting says "*inputs needed, result provided*". With annotations, it says "*takes an IP address (str) and a port number (int), returns True or False (bool)*". Much clearer for the next analyst reading your code.

Syntax

```
def function_name(param: type, param2: type) -> return_type:
    ...
```

Part	Example	Meaning
Parameter annotation	`ip: str`	`ip` should be a string
Return annotation	`-> bool`	the function returns a boolean

Part	Example	Meaning
No return value	`-> None`	the function returns nothing

Without annotations - ambiguous

In [ ]:
```
# Without annotations: what are ip and port supposed to be?
def check_port(ip, port):
    print(f"Scanning {ip}:{port}")
    return True   # but what does True mean here?
```

With annotations - self-documenting

In [ ]:
```
# With annotations: intent is crystal clear
def check_port(ip: str, port: int) -> bool:
    """Returns True if the port appears open, False otherwise."""
    print(f"Scanning {ip}:{port}")
    return True   # now the bool return makes sense


# More examples
def log_failed_login(username: str, source_ip: str, attempt_count: int) -> None:
    """Logs a failed authentication attempt. Returns nothing."""
    print(f"[ALERT] {attempt_count} failed login(s) for '{username}' from {source_i


def severity_label(cvss_score: float) -> str:
    """Converts a CVSS score (0.0-10.0) to a human-readable severity label."""
    if cvss_score >= 9.0:
        return "CRITICAL"
    elif cvss_score >= 7.0:
        return "HIGH"
    elif cvss_score >= 4.0:
        return "MEDIUM"
    return "LOW"


log_failed_login("admin", "10.0.0.99", 5)
print(severity_label(9.8))
print(severity_label(5.5))
```

Default parameter values with annotations

In [ ]:
```
# Default values work exactly as before - annotation goes before the default
def scan_target(
    ip: str,
    port: int = 80,
    timeout: float = 2.5,
    verbose: bool = False
) -> bool:
    """Attempts to connect to ip:port. Returns True if reachable."""
    if verbose:
```

```
        print(f"Scanning {ip}:{port} (timeout={timeout}s)")
    return True   # simplified - real version would use socket


scan_target("192.168.1.1")                        # uses all defaults
scan_target("192.168.1.1", port=443, verbose=True) # override some defaults
```

Practice - Section 3

Write: Add type annotations to the three functions below. Do not change the logic.

1. `is_internal_ip(ip)` - takes an IP address string, returns `True` if it starts with `10.` or `192.168.`, else `False`
2. `format_cve(year, seq_number)` - takes two integers, returns a formatted CVE string like `"CVE-2024-1234"`
3. `alert(message, level)` - takes a message string and an integer level (default 1), prints a formatted alert, returns nothing

In [ ]:
```
# Your code here - add annotations, keep the logic identical

def is_internal_ip(ip):
    return ip.startswith("10.") or ip.startswith("192.168.")


def format_cve(year, seq_number):
    return f"CVE-{year}-{seq_number}"


def alert(message, level=1):
    print(f"[LEVEL {level}] {message}")


print(is_internal_ip("192.168.1.5"))
print(is_internal_ip("8.8.8.8"))
print(format_cve(2024, 1234))
alert("Brute force detected", level=3)
```

▶ 💡 **SOLUTION**

Practice - Section 3 (Fix exercise)

Fix: The function below has broken annotations. Identify and fix all 3 errors.

In [ ]:
```
# Broken annotations - find and fix 3 errors

def classify_port(port: str, description: int) -> str:    # error 1
    """Returns a risk classification for a given port number."""
    risky_ports = [21, 23, 3389, 445]
    if port in risky_ports:
        return f"{description} is HIGH RISK"
    return f"{description} is LOW RISK"
```

```
def block_ip(ip: str, permanent: str = False) -> None:    # errors 2 and 3
    print(f"Blocking {ip} (permanent={permanent})")


# Your code here - fix the three annotation errors above
```

▶ 💡 **SOLUTION**

---

4. Complex Types: List, Dict, Tuple, Set

Analogy: Saying a variable is a `list` is like saying a filing cabinet contains "files". Useful, but what KIND of files? Incident reports? IP addresses? With complex type hints, you say: *"this is a list of IP addresses (strings)"* or *"this is a dict that maps port numbers (int) to service names (str)"*.

Python's `typing` module provides the tools to annotate complex collections.

Import from typing

```
from typing import List, Dict, Tuple, Set
```

> **Python 3.9+ shortcut:** You can write `list[str]` instead of `List[str]` (lowercase, no import needed).
> Both styles are valid. We'll use the `typing` import style so the code works on Python 3.7+.

Summary table

Type hint	What it means	Example
`List[str]`	A list where every item is a string	`["10.0.0.1", "10.0.0.2"]`
`List[int]`	A list where every item is an int	`[22, 80, 443]`
`Dict[str, float]`	A dict with string keys and float values	`{"CVE-2024-1234": 9.8}`
`Dict[int, str]`	A dict with int keys and string values	`{22: "SSH", 80: "HTTP"}`
`Tuple[str, int]`	A tuple of exactly (string, int)	`("192.168.1.1", 443)`
`Set[str]`	A set of unique strings	`{"10.0.0.1", "10.0.0.2"}`

In [ ]:
```
from typing import List, Dict, Tuple, Set


# List[str] - a list of IP addresses
blocked_ips: List[str] = ["10.0.0.55", "172.16.0.88", "192.168.1.200"]
```

```
# List[int] - a list of open ports
open_ports: List[int] = [22, 80, 443, 8080]

# Dict[int, str] - mapping port numbers to service names
port_services: Dict[int, str] = {
    22: "SSH",
    80: "HTTP",
    443: "HTTPS",
    3306: "MySQL",
    3389: "RDP"
}

# Dict[str, float] - CVE IDs mapped to their CVSS scores
vulnerabilities: Dict[str, float] = {
    "CVE-2024-1234": 9.8,
    "CVE-2023-9876": 7.5,
    "CVE-2022-5555": 4.2
}

# Set[str] - unique active attacker IPs (sets auto-deduplicate)
attacker_ips: Set[str] = {"203.0.113.5", "198.51.100.42"}

# Tuple[str, int] - a fixed (ip, port) pair representing a network endpoint
ssh_endpoint: Tuple[str, int] = ("10.0.0.1", 22)

print("Blocked IPs:", blocked_ips)
print("RDP service:", port_services[3389])
print("Top vuln CVSS:", vulnerabilities["CVE-2024-1234"])
print("SSH endpoint:", ssh_endpoint)
```

Complex types in function signatures

In [ ]:

```
from typing import List, Dict


def get_critical_cves(vulnerabilities: Dict[str, float], threshold: float = 9.0) ->
    """Returns a list of CVE IDs with a CVSS score at or above the threshold."""
    critical: List[str] = []
    for cve_id, score in vulnerabilities.items():
        if score >= threshold:
            critical.append(cve_id)
    return critical


def print_port_report(open_ports: List[int], port_services: Dict[int, str]) -> None
    """Prints a formatted report of open ports and their associated services."""
    print("Open Port Report")
    print("-" * 30)
    for port in open_ports:
        service = port_services.get(port, "Unknown")
        print(f"  Port {port:5d}  ->  {service}")


# Test get_critical_cves
```

```
vulns = {"CVE-2024-1234": 9.8, "CVE-2023-9876": 7.5, "CVE-2022-5555": 4.2}
critical_list = get_critical_cves(vulns)
print("Critical CVEs:", critical_list)

# Test print_port_report
ports = [22, 80, 443, 3389]
services = {22: "SSH", 80: "HTTP", 443: "HTTPS", 3389: "RDP"}
print_port_report(ports, services)
```

Practice - Section 4

Write: Complete the function `summarise_scan` below.

- It takes `scan_results` - a dictionary where keys are IP addresses (str) and values are lists of open port numbers (List[int])
- It returns the total number of open ports found across all hosts (int)
- Add complete type annotations to the function signature

Example: `{"10.0.0.1": [22, 80], "10.0.0.2": [443]}` should return `3`

In [ ]:
```
from typing import Dict, List

# Your code here
def summarise_scan(scan_results):
    pass


# Test
results = {
    "10.0.0.1": [22, 80, 8080],
    "10.0.0.2": [443],
    "10.0.0.3": [22, 3306]
}
print(summarise_scan(results))  # Expected: 6
```

▶ 💡 **SOLUTION**

---

5. Optional - Values That Can Be None

Analogy: Imagine a lookup in your vulnerability database. You search for `CVE-2024-9999`. It might return a full record - or it might return nothing because the CVE doesn't exist yet. That "nothing" in Python is `None`.

`Optional[X]` is shorthand for *"this value is either of type X, or it's None."*

Syntax

```python
from typing import Optional

variable: Optional[str] = None      # either a string, or None

def lookup(cve_id: str) -> Optional[float]:  # returns a float OR None
    ...
```

When to use Optional

Situation	Type hint
A lookup that may find nothing	`-> Optional[str]`
A parameter that has a `None` default	`param: Optional[str] = None`
A variable set later in a class `__init__`	`self.result: Optional[dict] = None`

In [ ]:
```python
from typing import Optional, Dict


def lookup_cvss(cve_id: str, database: Dict[str, float]) -> Optional[float]:
    """
    Returns the CVSS score for a CVE ID, or None if it's not in the database.
    The Optional[float] return type signals to the caller: always check for None!
    """
    return database.get(cve_id)   # .get() returns None if key not found


def get_last_seen(ip: str, activity_log: Dict[str, str]) -> Optional[str]:
    """Returns the last-seen timestamp for an IP, or None if never logged."""
    return activity_log.get(ip)


# Usage
vuln_db: Dict[str, float] = {"CVE-2024-1234": 9.8, "CVE-2023-9876": 7.5}

score = lookup_cvss("CVE-2024-1234", vuln_db)
if score is not None:
    print(f"CVSS score: {score}")

score = lookup_cvss("CVE-2099-9999", vuln_db)
if score is None:
    print("CVE not found in database")
```

Optional parameters (default None)

In [ ]:
```python
from typing import Optional


def create_alert(
    message: str,
    source_ip: Optional[str] = None,    # ip is optional - alert may have no source
    ticket_id: Optional[str] = None     # may not be linked to a ticket yet
```

```
) -> str:
    """Formats an alert message, optionally including source IP and ticket referenc
    alert = f"[ALERT] {message}"
    if source_ip is not None:
        alert += f" | Source: {source_ip}"
    if ticket_id is not None:
        alert += f" | Ticket: {ticket_id}"
    return alert


# With all fields
print(create_alert("Port scan detected", "203.0.113.5", "INC-0042"))
# With only the message
print(create_alert("Config drift detected"))
```

Practice - Section 5

Write: A function `find_open_port` that:

- Takes a list of port numbers (`List[int]`) and a target port (`int`)
- Returns the port number if found in the list, or `None` if not
- Use `Optional` in the return annotation
- Call it twice and handle both the found and not-found cases

In [ ]:
```
from typing import List, Optional

# Your code here
```

▶ 💡 **SOLUTION**

---

6. Union - Values That Can Be Multiple Types

Analogy: Some fields in a security report can hold different types of data depending on the context. An alert source might be identified by an IP address (string) or a port number (integer). `Union` lets you say exactly that.

`Union[X, Y]` means "*this value is either type X or type Y.*"

Syntax

```
from typing import Union

identifier: Union[str, int] = "10.0.0.1"   # could be an IP or a port
number
```

> **Python 3.10+ shortcut:** `str | int` instead of `Union[str, int]`.

Important: Union is not the same as Optional

Situation	Correct hint
Value is str OR int	`Union[str, int]`
Value is str OR None	`Optional[str]` (or `Union[str, None]`)
Value is str, int, OR None	`Union[str, int, None]`

In [ ]:
```
from typing import Union


def format_identifier(identifier: Union[str, int]) -> str:
    """
    Formats a network identifier for display.
    Accepts either an IP address (str) or a port number (int).
    """
    if isinstance(identifier, int):
        return f"PORT:{identifier}"
    return f"IP:{identifier}"


print(format_identifier("192.168.1.5"))  # IP:192.168.1.5
print(format_identifier(443))             # PORT:443


# Union in variable annotations
alert_source: Union[str, int] = "10.0.0.55"  # starts as IP
print(format_identifier(alert_source))

alert_source = 3389                           # now a port number
print(format_identifier(alert_source))
```

In [ ]:
```
from typing import Union, Dict, List


# Real-world scenario: a log parser that returns different types
# depending on the log line content
def parse_log_value(raw_value: str) -> Union[int, float, str]:
    """
    Tries to parse a raw log field as int, then float, then returns as str.
    Common pattern when reading heterogeneous log files.
    """
    try:
        return int(raw_value)       # e.g. "443" -> 443
    except ValueError:
        pass
    try:
        return float(raw_value)     # e.g. "9.8" -> 9.8
    except ValueError:
        pass
    return raw_value                # e.g. "CRITICAL" -> "CRITICAL"
```

```
log_fields = ["443", "9.8", "CRITICAL", "192.168.1.1"]
for field in log_fields:
    parsed = parse_log_value(field)
    print(f"{field!r:20} -> {parsed!r:20} ({type(parsed).__name__})")
```

Practice - Section 6

Write: A function `describe_threat` that:

- Takes `threat` as a `Union[str, int]`
- If it's an `int`: assumes it's a CVSS score, returns `"CVSS {score} - {label}"` where label is CRITICAL/HIGH/MEDIUM/LOW
- If it's a `str`: assumes it's a CVE ID, returns `"CVE reference: {threat}"`
- Returns a string in both cases

In [ ]:
```
from typing import Union

# Your code here
```

▶ 💡 **SOLUTION**

---

7. Type Aliases

Analogy: In security documentation, instead of writing *"a dictionary mapping string CVE IDs to floating-point CVSS scores"* every single time, you'd define a shorthand and call it a "vulnerability database". Type aliases do exactly this for your code.

Syntax

```
# Create the alias
AliasName = SomeComplexType

# Use it just like any other type
variable: AliasName = ...
def func(param: AliasName) -> AliasName:
    ...
```

When to use type aliases

- When the same complex type appears in multiple places
- When the type hint is long and reduces readability
- When the alias name makes the code's domain clearer

In [ ]:
```
from typing import Dict, List, Tuple

# --- Define aliases ---
```

```python
# An IP address is just a string, but naming it clarifies intent
IPAddress = str

# A vulnerability database: CVE ID -> CVSS score
VulnDatabase = Dict[str, float]

# A scan result: IP -> list of open ports
ScanResult = Dict[IPAddress, List[int]]

# A network endpoint: (ip, port) pair
Endpoint = Tuple[IPAddress, int]


# --- Now use the aliases in function signatures ---

def get_critical_hosts(scan: ScanResult, vuln_db: VulnDatabase) -> List[IPAddress]:
    """
    Returns IPs that have port 3389 (RDP) open - a common attack vector.
    This function signature is readable BECAUSE of the aliases.
    Without them: Dict[str, List[int]], Dict[str, float] -> List[str]
    """
    rdp_port = 3389
    at_risk: List[IPAddress] = []
    for ip, ports in scan.items():
        if rdp_port in ports:
            at_risk.append(ip)
    return at_risk


# Test
scan_data: ScanResult = {
    "10.0.0.1": [22, 443],
    "10.0.0.2": [22, 3389, 445],   # RDP exposed!
    "10.0.0.3": [80, 3389],         # RDP exposed!
}

risky_hosts = get_critical_hosts(scan_data, {})
print("Hosts with RDP exposed:", risky_hosts)
```

Practice - Section 7

Write: Define the following type aliases and use them in a function.

- `LogEntry` - a string (one raw log line)
- `LogFile` - a list of `LogEntry` items
- `ParsedLog` - a dict mapping a timestamp string to a list of `LogEntry` items grouped by that timestamp

Write a function `count_entries(log: LogFile) -> int` that returns the number of entries in the log.

In [ ]:
```
from typing import Dict, List

# Your code here - define aliases and write the function
```

▶ 💡 **SOLUTION**

---

8. Why Type Hints Matter in Cybersecurity Code

Type hints are especially valuable in security tooling for four reasons:

1. Code documentation (self-documenting functions)

Security code is often read by team members who didn't write it. An annotated function tells you everything without reading the body:

```
# What does this take? What does it return?
def scan(t, p, v): ...

# This is immediately clear:
def scan(target: str, ports: List[int], verbose: bool = False) ->
Dict[int, bool]: ...
```

2. IDE support

With type hints, your editor (VS Code, PyCharm) offers:

- Auto-complete specific to the correct type
- Inline warnings when you pass the wrong type
- Hover documentation showing what a function expects

3. Static analysis with mypy

`mypy` is a tool that reads your code and checks type annotations **before you run anything**. Install it with `pip install mypy`.

```
# Run mypy on your scanner script
mypy port_scanner.py
```

If you write `check_port("10.0.0.1", "443")` but the annotation says `port: int`, mypy will catch it immediately and tell you: `Argument 2 has incompatible type "str"; expected "int"`

4. Cleaner architecture

Writing annotations forces you to think: "*what exactly does this function need? What exactly does it return?*" This leads to smaller, cleaner, more testable functions - which matters a lot in

security tooling where bugs can have real consequences.

Check Your Understanding: When Would You Use Each?

Scenario	Correct hint to use
A list of open ports	`List[int]`
A port-to-service map	`Dict[int, str]`
A lookup that might return nothing	`Optional[str]`
A field that's either an IP or a port	`Union[str, int]`
Shorthand for a complex repeated type	Type alias
A function that returns nothing	`-> None`

In [ ]:
```
# Putting it all together: a typed network security tool skeleton
# This is what your final Network Security Tool project functions should look like

from typing import Dict, List, Optional, Tuple

# Type aliases for readability
IPAddress = str
Port = int
ServiceMap = Dict[Port, str]
ScanResult = Dict[IPAddress, List[Port]]

KNOWN_SERVICES: ServiceMap = {
    21: "FTP",
    22: "SSH",
    23: "Telnet",
    80: "HTTP",
    443: "HTTPS",
    3306: "MySQL",
    3389: "RDP",
    8080: "HTTP-Alt"
}

DANGEROUS_PORTS: List[Port] = [21, 23, 3389, 445]  # FTP, Telnet, RDP, SMB


def get_service_name(port: Port, service_map: ServiceMap) -> str:
    """Returns the service name for a port, or 'Unknown' if not in the map."""
    return service_map.get(port, "Unknown")


def is_dangerous(port: Port, dangerous_ports: List[Port]) -> bool:
    """Returns True if the port is on the list of high-risk ports."""
    return port in dangerous_ports


def format_scan_report(scan: ScanResult, service_map: ServiceMap) -> Optional[str]:
```

```
    """Returns a formatted report string, or None if the scan found nothing."""
    if not scan:
        return None
    lines: List[str] = ["=== SCAN REPORT ==="]
    for ip, ports in scan.items():
        lines.append(f"Host: {ip}")
        for port in ports:
            service = get_service_name(port, service_map)
            danger = " [!!! DANGEROUS]" if is_dangerous(port, DANGEROUS_PORTS) else
            lines.append(f"  {port:5d}  {service}{danger}")
    return "\n".join(lines)


# Demo scan result
demo_scan: ScanResult = {
    "10.0.0.1": [22, 80, 443],
    "10.0.0.2": [22, 23, 3389],  # two dangerous ports!
}

report = format_scan_report(demo_scan, KNOWN_SERVICES)
if report is not None:
    print(report)
```

---

9. Summary

Concept	Syntax	Use when
Variable annotation	`ip: str = "10.0.0.1"`	Documenting a variable's expected type
Function parameter	`def f(ip: str, port: int):`	Making function contracts explicit
Return type	`def f() -> bool:`	Showing what a function returns
No return	`def f() -> None:`	Function only has side effects (prints, writes)
List	`List[int]`	A list of items of one type
Dict	`Dict[str, float]`	A dict with typed keys and values
Tuple	`Tuple[str, int]`	A fixed-size, immutable ordered pair
Set	`Set[str]`	A set of unique items
Optional	`Optional[str]`	Value is str OR None
Union	`Union[str, int]`	Value can be multiple types
Type alias	`VulnDB = Dict[str, float]`	Shorthand for a complex repeated type

Key rules to remember

1. **Annotations are hints, not enforcement** - Python won't crash at runtime if you use the wrong type. Use `mypy` to catch errors statically.
2. **Import complex types from `typing`** - `List`, `Dict`, `Tuple`, `Set`, `Optional`, `Union`.
3. **`Optional[X] = Union[X, None]`** - they are equivalent.
4. **Always check for `None`** when calling a function that returns `Optional[X]`.
5. **Type aliases improve readability** - name your types to reflect the security domain.

Next Steps

- Complete **01_Drills_Typing.ipynb** - 15 exercises to consolidate everything